

El proyecto final de la materia, denominado el "proyecto mayor", tiene un valor del **20%** de la calificación total. Este proyecto está diseñado para orientar a los estudiantes hacia el **cómputo científico** y el uso de **sistemas distribuidos**.

A continuación, se detallan las instrucciones, los objetivos y las herramientas requeridas y recomendadas para la realización del proyecto:

1. ¿Qué es el Proyecto yCuál es su Objetivo?

El proyecto es la implementación de un **sistema de recomendación distribuido** para trabajar con *big data* (grandes volúmenes de datos).

El objetivo principal es **calcular la matriz de similitudes de 100 libros** provenientes del proyecto Gutenberg.

El sistema resultante debe proporcionar tres funciones principales:

1. **Cálculo de la Matriz de Similitudes (\$\$\$)**: Una matriz que indique qué tan similares son los documentos entre sí.
2. **Sistema de Recomendación**: Una función que, dado un libro de entrada, recomiende los **10 libros más similares**.
3. **Resumen de Documentos**: Una función que describa un libro al proporcionar los **10 términos más importantes** del mismo.

2. Tareas Clave y Algoritmo a Implementar

La base del proyecto es la implementación de un algoritmo mediante el paradigma de **Map-Reduce**. El proceso se centra en el cálculo de **TF-IDF** (Frecuencia de Término - Frecuencia Inversa de Documento) y la **Similitud por Cosenos**.

El pseudocódigo implica los siguientes pasos lógicos:

A. Preprocesamiento (Limpieza de Documentos)

Antes de cualquier cálculo, se debe limpiar el *corpus* de documentos para asegurar la coherencia:

- Convertir todo a **minúsculas**.
- **Limpiar caracteres especiales** y signos de puntuación.
- **Tokenizar** (transformar el texto en una lista de palabras).
- **Eliminar Stop Words** (palabras comunes que no ayudan a diferenciar documentos, como pronombres o verbos auxiliares).

B. Estructura de Datos de Entrada

Los 100 libros deben cargarse en la memoria del sistema distribuido. La entrada debe representarse como tuplas de **llave y valor**:

- **Llave:** El nombre del documento (ej., `doc_i`).
- **Valor:** El texto completo o la lista de todas las palabras de ese libro.
- **Sugerencia de Carga:** Se recomienda utilizar el comando de lectura que carga **cada libro completo como un solo elemento** (generando 100 elementos) en lugar de leer línea por línea.

C. Cómputo con Map-Reduce (Cálculo TF y DF)

El proceso central de Map-Reduce (Mapa, Agrupamiento y Reducción) se utiliza en dos fases:

1. **Cálculo de Frecuencia de Término (TF):** Para determinar cuántas veces aparece una palabra en un documento específico, normalizado por el tamaño total de ese documento.
 - **Mapa:** Transformar la entrada (`doc_i`, [`palabras`]) en múltiples pares de llave-valor: (`(doc_i, palabra_j)`, 1).
 - **Reducción:** Agrupar por la llave (`doc_i, palabra_j`) y aplicar la función **suma** a los valores (1, 1, 1...) para obtener la frecuencia de la palabra en el documento. Luego, normalizar este valor para obtener el **TF**.
2. **Cálculo de Frecuencia Inversa de Documento (IDF):** Para determinar la importancia de una palabra en todo el *corpus* (penalizar las palabras muy comunes).
 - **Mapa:** Transformar los resultados de TF en pares (`palabra_j`, 1).
 - **Reducción:** Agrupar por la llave (`palabra_j`) y aplicar la función **suma** para contar en cuántos documentos aparece la palabra, obteniendo la **DF**.

D. Transformación a Vectores y Similitud

Una vez obtenidos los valores de TF y DF, se calcula el peso final (\$W\$) de cada palabra en cada documento mediante la fórmula

$$W_{i,j} = TF_{i,j} \cdot \log \left(\frac{N}{DF_i} \right).$$

Estos pesos forman los **vectores** que representan cada documento.

Finalmente, la **Similitud por Cosenos** se aplica a los vectores de los documentos (U y W) para construir la matriz de similitudes ($S_{u,w}$), donde

$$S_{u,w} = \cos(\theta_{u,w}) = \frac{U \cdot W}{\|U\| \cdot \|W\|}.$$

3. Herramientas Requeridas y Recomendadas

El proyecto requiere el uso de herramientas específicas del **cómputo distribuido** y prácticas de **programación profesional**.

Categoría	Herramienta/Tecnología	Detalles importantes
Plataforma Distribuida	Apache Spark (PySpark)	Es la plataforma que se utilizará para el cómputo. El proyecto debe implementarse utilizando el módulo de Python (PySpark).
Lenguaje de Programación	Python	Es el lenguaje preferido para la implementación.
Estructura de Datos	DataFrames (Recomendado)	Aunque los RDD (Resilient Distributed Datasets) son la base, se recomienda encarecidamente usar DataFrames, ya que son más rápidos, son tratados como tablas (similares a SQL) y facilitan la optimización de operaciones.
Cómputo Científico	GPU / Álgebra Lineal	La implementación debe utilizar operaciones eficientes para la multiplicación de matrices (que es la base de la similitud por cosenos). Se menciona el concepto de valores propios como parte de los temas involucrados.
Entorno de Trabajo	Google Colab (Recomendado)	No es necesario comprar una tarjeta gráfica (GPU) . Se puede usar Google Colab para ejecutar y probar el código aprovechando las tarjetas gráficas que proporciona Google de forma gratuita. También se puede usar Jupyter Lab en el ambiente virtual local.
Reporte Final	LaTeX / Overleaf (Requerido)	Los reportes técnicos del curso deben entregarse utilizando escritura profesional en LaTeX . Se recomienda la plataforma Overleaf para esto.
Gestión de Código	Repositorios (GitHub/Bitbucket) (Requerido)	Es fundamental dominar el uso de repositorios para la colaboración y como parte de un portafolio profesional.
Práctica de Código	Documentación de Funciones (Recomendado)	Se enfatiza la importancia de una documentación completa y de calidad (docstrings), siguiendo estándares como el formato NumPy (ej. detallando <code>Parameters</code> y <code>Returns</code>).

Entrega: El proyecto se debe entregar en **grupos de máximo tres personas**. El plazo de entrega se extendió hasta el **lunes** de la siguiente semana.

COMPLEMENTO PROPORCIONADO POR EL PROFESOR

get_books.py

```
import re
import requests

from bs4 import BeautifulSoup
from concurrent.futures import ThreadPoolExecutor

def get_links(n: int | list[int] = -1) -> tuple[ list[str], list[str] ]:
    """Gets the urls for books in the Gutenberg project with numbers `n`.

    The books are in txt format under the section frequently downloaded in:
        https://www.gutenberg.org/browse/scores/top.

    The numbers `n` must correspond to those on this list (starting with
    one).

    Parameters
    -----
    n : int | list[int], optional
        An integer or list of integers with the desired book numbers. Choose
        -1
        (default) if all books are desired.

    Returns
    -----
    links : list[str]
        Links to the txt files of the books.
    titles : list[str]
        Book titles.
    """
    assert n == -1 or min(n) > 0, ( "n can only take values greater than
    zero "
                                   "or -1 if all links are desired" )

    # Get the html of the top books in Gutenberg project
    url = "https://www.gutenberg.org/browse/scores/top"
    try:
        response = requests.get(url)

        # Parse contents with BeautifulSoup
        parser = BeautifulSoup(response.text, 'html.parser')

        # Get links of books
```

```
ordered_list = parser.find('ol') # get first ordered list
list_items = ordered_list.find_all('li') # list items inside
if(n != -1):
    list_filtered = [list_items[i-1] for i in list(n)]
else:
    list_filtered = list_items

prefix = "https://www.gutenberg.org"
suffix = ".txt.utf-8"
links, titles = [], []
for li in list_filtered:
    links.append(prefix + li.find("a").get("href") + suffix)
    # replace whitespaces by underscores
    title = re.sub( r'\s+', '_', li.get_text())
    # remove numbers and parenthesis at end of filename
    title = re.sub(r'_\(\d+\)$', '', title)
    title += r'.txt' # add file extension
    titles.append(title)
return links, titles

except requests.exceptions.RequestException as e:
    print("wrong url for Gutenberg project")

def download_file(url, name):
    response = requests.get(url, stream=True)
    with open(name, mode='wb') as file:
        for chunk in response.iter_content(chunk_size=10 * 1024): #10kb
            file.write(chunk)
    print(f"Downloaded file: {name}")

def store_files(links, names):
    with ThreadPoolExecutor() as executor:
        executor.map(download_file, links, names)

def store_files_slow(links, names):
    for url, name in zip(links, names):
        download_file(url, name)

def main( n = -1 ):
    links, titles = get_links(n)
    store_files(links, titles)
    print("Done")

n = range(1, 41)
```

Proyecto Sistemas Distribuidos

```
main( n )
```